AMATH 582: TOPOLOGICAL DATA ANALYSIS AND MANIFOLD LEARNING: SPECTRAL APPROACH TO CONGRESSIONAL VOTING RECORDS

PARTHA KRISHNAMURTHY

Applied And Computational Mathematics, University of Washington, Seattle, WA
ipartha2@uw.edu

1. Introduction and Overview

Real world data is highly non-convex. Naive clustering methods such as *k-Means* enforce strict spherical boundaries, failing to respect the underlying continuous topology of complex manifolds. In this paper I explore the usage of manifold learning to classify Congressional members based on their voting records. The core engine that drives the clustering algorithm is the Laplacian embeddings that are constructed in a lower dimensional *eigenspace*. The embeddings are then reused to learn the political affiliation of members using only a subset of data. I empirically show that this semi-supervised regression algorithm is highly dependent on the dimensionality of the embeddings and the number of data points that are used in the learning.

2. Theoretical Background

Before we get started on the theoretical foundation of the Spectral Clustering algorithm, I begin this section by showing the failure of *k-Means* clustering on the data.

2.1. *k-Means* clustering and its failure on non-linear manifolds.

The idea behind *k-Means* clustering is to group data points based on their proximity to a set of K clusters.

For a dataset $\{x_i\}_{i=1}^N$ where $x_i \in \mathbb{R}^d$ for $i = 1, \ldots, N$, we consider measuring distances between the data points. Then the goal of *k-Means* clustering is to find

- a set of cluster centers $\{m_k\}_{k=1}^K$ where $m_k \in \mathbb{R}^d$ for $k = 1, \ldots, K$
- a partitioning of $\mathbf{X}$ into K groups, X_k for $k = 1, \ldots, K$ such that $\bigcup_{k=1}^K X_k = X$ and $X_k \cap X_l = \emptyset$ for $k \neq l$.

The quality of the clustering is measured by computing the sum of squared distances from each data point to its assigned cluster.

$$SSD(k) = \sum_{x_i \in X_k} ||x_i - m_k||^2 \tag{1}$$

Then the total sum of squared distances is given by

$$SSD(\{X_k\}, \{m_k\}_{k=1}^K) = \sum_{k=1}^K SSD(k) \tag{2}$$

The goal of the *k-Means* algorithm is to minimize this objective function.

Although the *k-Means* algorithm achieved comparable accuracy results with advanced Spectral clustering methods, a deeper analysis using **t-SNE** methods revealed the flaw with this approach. While the method is simple, *k-Means* relies on Euclidean distances to define the clusters. When the data lies on non-linear manifold, then the algorithm will fail to assign the data points to appropriate clusters. Figure 1 uses t-SNE

Date: 03/14/2026.

to visualize the true local neighborhoods of the 16D data, revealing how the rigid Euclidean boundaries of *k-Means* artificially sever continuous communities (e.g., the isolated points at coordinates 5, -5).

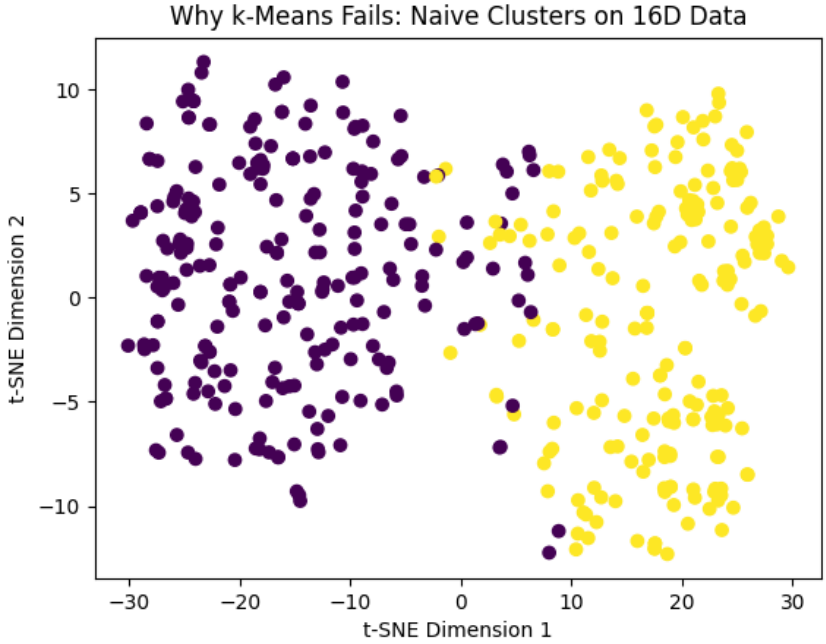


FIGURE 1. *k-Means* failure on mislabeled data at coordinates (5, -5)

The details of the clustering in higher dimensional methods will be covered in future sections.

2.2. **Graph Laplacians.** The core building block of Spectral clustering is the construction of a feature map Ψ. The standard *k-Means* algorithm can be applied to $\Psi(X)$ and that will help in overcoming the issues presented when performing clustering on non-linear geometries and achieve better clustering performance.

Given the data set $X = \{x_1, \ldots, x_N\}$, the first step is to build a graph $G = \{X, W\}$ with vertices X and edge weights W. $W \in \mathbb{R}^{N\times N}$ is a symmetric matrix with non-negative entries $w_{ij} \geq 0$ and indicates the strength of connection between the data points x_i and x_j. The matrix W is sometimes referred to as *proximity matrix* of the graph G, where the entry w_{ij} is large if x_i and x_j are close to each other and small if they are far apart. If there are no connections between x_i and x_j, then $w_{ij} = 0$.

The degree matrix D of the graph G is defined as,

$$D = diag(d_1, \ldots, d_N), \quad d_i = \sum_{j=1}^{N} w_{ij}$$

The vertices with larger d_i or higher degree have more edges with larger weights connecting to them and are categorized as "more important".

The matrices W and D are discrete analogues of inhomogeneous Laplacian operators of the form,

$$\mathcal{L}f = -\frac{1}{\rho}\nabla \cdot (\rho^2 \nabla f)$$

where $\rho \in \mathbb{R}^d$ is the distribution.

Three common choices of Laplacians that are used in Spectral clustering

- Unnormalized Graph Laplacian: $L_{UN} = D - W$
- Normalized Graph Laplacian: $L_N = D^{-\frac{1}{2}}(D - W)D^{-\frac{1}{2}}$

- Random walk Graph Laplacian: $L_{RW} = D^{-1}(D - W)$

For the purposes of Spectral Clustering, the key property of interest is the relationship between the eigendecomposition of L and the geometry of the underlying graph G.

2.3. **Diffusion maps.** The unnormalized and the normalized Laplacians are positive semidefinite matrices i.e all eigenvalues of the Laplacians are ≥ 0. And they are also guaranteed to have an eigenvalue at zero. It can be analytically proven that if the graph G is connected, then the zero eigenvalue is simple, i.e., it has a multiplicity of one. On the other hand, if the graph is not connected, then the multiplicity of the zero eigenvalue is equal to the number of connected components.

The above observation about the spectral properties of Graph Laplacians motivates the definition of a feature map that can reveal the clustering structure of the data. The eigen decomposition of the Laplacian is given by,

$$L = U\Lambda U^T$$

with the eigenpairs of L ordered in ascending order of the eigenvalues i.e $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N$. Then the diffusion feature map $\Psi_t : X \to \mathbb{R}^N$ can be defined as

$$\Psi_t(x_i) = ((\tau + \lambda_1)^{-t} u_1(i), \ldots, (\tau + \lambda_N)^{-t} u_N(i))^T$$

where $u_j(i)$ is the i-th entry of the j-th eigenvector of L and $\tau > 0$ is an epsilon to avoid division by zero and t is the diffusion time and the hyperparameter is chosen by the user.

Then diffusion feature maps define a kernel on X,

$$K_t(x_i, x_j) = \Psi_t(x_i)^T \Psi_t(x_j) = \sum_{k=1}^{N} (\tau + \lambda_k)^{-2t} u_k(i) u_k(j).$$

In practice, standard spectral clustering algorithms utilize an unweighted, truncated version of this map, dropping the eigenvalue scaling entirely and retaining only the first $K + M$ eigenvectors. This simple feature map can be written as,

$$\Psi_{Lap}(x_i) = (u_1(i), \ldots, u_{K+M}(i))^T$$

.

The *k-means* clustering can then be applied to the transformed dataset $Z = \Psi(X) = \{\Psi(x_1), \ldots, \Psi(x_N)\}$ to cluster the data points in X.

2.4. **Semi-Supervised Learning (SSL).** While in supervised learning the training data is of the form $\{x_i, y_i\}_{i=1}^N$ and in unsupervised learning the data is of the form $\{x_i\}_{i=1}^N$, semi-supervised learning has a combination of both and is of the form,

$$\{x_i, y_i\}_{i=1}^M \cup \{x_i\}_{i=M+1}^N.$$

The goal of **SSL** is to leverage the geometry of the data to find the labels for $\{x_i\}_{i=M+1}^N$.

2.4.1. *Semi-supervised regression (SSR).* Semi-supervised Regression takes advantage of the geometry of the data to define a feature map. Consider a graph $G = \{X, W\}$ built on the data and let $u_1, \ldots, u_J \in \mathbb{R}^N$ be the first J eigenvectors of graph Laplacian matrix defined on G. Then the model class can be defined as,

$$\mathcal{F} = \left\{ f : X \to \mathbb{R} | f(x) = \sum_{j=1}^{J} c_j u_j(x) \right\}$$

where the eigenvector u_j is viewed as a function defined on X such that $u_j(x_i)$ corresponds to the i-th entry of the vector.

Then a regressor is computed on the labeled data that can be used to determine the labels to the rest of the data. In particular, we solve the following optimization problem

$$\hat{f} = \sum_{j=1}^{J} \hat{c_j} u_j, \quad \hat{c} = \arg\min_{c \in \mathbb{R}^J} \sum_{i=1}^{M} \left(y_i - \sum_{j=1}^{J} c_j u_j(i) \right)^2$$

2.4.2. *Semi-supervised classification(SSC).* The Semi-supervised Regression problem can be extended to Semi-supervised Classification problem by introducing a thresholding step. For multi-class classification, we use one-hot encoding to encode the labels and the above regression is applied for each class separately. For example, on a k-class classification problem, we consider $k = 1, \ldots, K$ independent regressors of the form,

$$\hat{f_k} = \sum_{j=1}^{J} \hat{c_{jk}} u_j, \quad \hat{c}_k = \arg\min_{c \in \mathbb{R}^J} \sum_{i=1}^{M} \left(y_{ik} - \sum_{j=1}^{J} c_j u_j(i) \right)^2$$

3. Algorithm Implementation and Development

This section details the algorithms that were developed based on the theoretical foundations established in Theoretical Background 2 section to cluster politicians. The voting records of members of the the U.S. House of representatives across 16 bills were utilized as the primary dataset X, with true political affiliations mapped to binary labels $y \in \{-1, 1\}$. The unsupervised algorithm partitions the underlying manifold based strictly on topological voting patterns and its empirical accuracy is evaluated directly against these ground-truth labels.

3.1. **Data loading and pre-processing.** Raw datasets are rarely used to train Machine Learning models. The voting records dataset is built with characters *y* and *n* and the political affiliations mention the *two* major U.S. parties - *republican* or *democrat*. In order to formulate the data as a continuous feature matrix $X \in \mathbb{R}^{N \times d}$, the raw dataset is preprocessed by assigning unique values per voting choice and party affiliation. To minimize memory overhead, an on-the-fly mapping function is applied during data ingestion to instantly encode the categorical variables.

Algorithm 1 Data loading and pre-processing

Require: Raw data file, house-votes-84.data
Ensure: Processed dataset, $data \in \mathbb{R}^{N \times d}$

1: Define party_converter lambda that assigns 1 if values of a column in raw data is 'republican' and -1 otherwise
2: Define party_converter lambda that assigns 1 if values of a column in raw data is 'y' or -1 for 'n' and 0 otherwise.
3: Initialize $converters$ mapping indices to converter lambdas.
4: Assign index-0 to party_converter
5: Assign indices 1 through 16 to party_converter
6: Load the raw data file by passing the converter, $data \leftarrow$ LOADTXT(house-votes-84.data, converters) ▷ Passing converters ensures the resultant $data$ matrix is preprocessed and assigned appropriate numerical values.
7: **return** data

The algorithm 1 assigns *0* values to missing votes. This ensures the missing data is placed equi-distant from the two ends of the feature space $[-1, 1]$. This ensures missing data doesn't artificially skew the pairwise distance calculations.

3.2. **Clustering.** This unsupervised clustering algorithm is built using multiple sub-routines. In this section we will cover each subroutine and the final main routine that evaluates the performance of the clustering by comparing the raw counts of matches with ground-truth labels.

3.2.1. *Building Weight Function and the Degree matrix.* The Weight function is a kernel matrix that is built by calculating the distance between each data point. The degree is the sum of the weights computed for each data point.

Algorithm 2 Weight Function and Degree Matrix

Require: The data matrix, $X \in \mathbb{R}^{N \times d}$, Gaussian width σ
Ensure: Weight function, W and the degree matrix, D
1: Obtain the difference for each pair of data points, $diff \leftarrow X[:, None, :] - X[None, :, :]$ ▷ Vectorization method by broadcasting different indices and subtracting every pair of points
2: Calculate the squared norm of the differences, $norm \leftarrow (\text{NORM}(diff))^2$
3: Calculate Weight Kernel, $W \leftarrow \text{EXPONENTIAL}(-\frac{norm}{2*\sigma^2})$
4: Set the diagonal values to be zero, $W[range(W.shape[0]), range(W.shape[0])] \leftarrow 0.0$ ▷ This will ensure the distance between points with themselves will not be accounted.
5: Calculate the vector of sums of each rows of matrix W, $s \leftarrow \text{SUM}(W, axis = 1)$
6: Calculate degree matrix by diagonalizing the sum matrix, $D \leftarrow \text{DIAG}(s)$
7: **return** W, D

3.2.2. *Laplacian Embedding.* This method computes the un-normialized Laplacian by constructing the Weight function and Degree matrix by invoking the 2 algorithm. The function returns the eigen vectors of the Laplacian matrix.

Algorithm 3 Laplacian Embedding

Require: The data matrix, $X \in \mathbb{R}^{N \times d}$, Gaussian width σ
Ensure: Eigen vector of the laplacian, v
1: Calculate the Weight function and Degree matrix, $W, D \leftarrow$ WeightFunctionAndDegreeMatrixCalculator(X, σ) ▷ This invokes the algorithm 2.
2: Calculate Laplacian, $L \leftarrow D - W$
3: Calculate eigens of L, $w, v \leftarrow \text{EIGH}(L)$ ▷ eigh is a function provided by numpy that returns the eigen vectors of a matrix sorted in ascending order of eigen values
4: **return** v

3.2.3. *Clustering.* This is the core algorithm that calculates the Fiedler vector from the Laplacian embeddings and clusters the data by bi partitioning the embeddings. The performance of the clustering is determined by comparing the assigned label against the ground-truth labels.

Note: The unsupervised algorithm operates blindly regarding the political affiliation, assigning clusters based purely on the topological connectivity captured by the Fiedler vector's zero-crossing. In order to adjust for this, we take the misclassified labels as the minimum of the matched and mismatched labels.

3.3. **Semi-supervised Regression.** As described in theoretical section 2, the Semi-supervised regression algorithm uses the eigen vectors of the Graph Laplacian matrix to build a regressor on labeled data. This regressor is used to determine the labels to the rest of the data.

The unsupervised clustering algorithm provided us with an optimal Gaussian width σ^* where the bipartitioning achieved maximum accuracy. We will use the optimal σ value when computing the Laplacian embeddings required for building the regressor.

Algorithm 4 Clustering

Require: The data matrix, $X \in \mathbb{R}^{N\times d}$, Ground-truth labels $Y \in \mathbb{R}^N$, Gaussian width σ
Ensure: Clustering accuracy, $accuracy$
1: Get eigen vectors of laplacian embedding matrix $e \leftarrow \text{LaplacianEmbedding}(X, \sigma)$
2: Build Fiedler vector, $q_1 = e[:, 1]$
3: Calculate predicted clustering class, $\hat{y} \leftarrow \text{sign}(q_1)$
4: Calculate matched and mismatched values, $match, mismatch \leftarrow \text{sum}(y == \hat{y}), \text{sum}(y \neq \hat{y})$
5: Calculate actual mismatches, actual_mismatch $\leftarrow \text{min}(match, mismatch)$
6: Calculate accuracy $accuracy \leftarrow 1 - (\frac{\text{actual_mismatch}}{N})$
7: **return** $accuracy$

Algorithm 5 Normal Equation Solver

Require: Data $A \in \mathbb{R}^{L\times M}$, Ground truth labels $b \in \mathbb{R}^L$
Ensure: Solution to normal equation, $\hat{\beta}$
1: at_a $\leftarrow$ matrix_multiplication(A^T, A)
2: at_b $\leftarrow$ matrix_multiplication(A^T, b)
3: at_a_inv $\leftarrow$ inverse(at_a)
4: $\hat{\beta} \leftarrow$ matrix_multiplication(at_a_inv, at_b)
5: **return** $\hat{\beta}$

3.3.1. *Normal equation solver.* The regression algorithm requires a normal equation solver that finds the minimizer to a least square problem.

3.3.2. *Semi-supervised regression.* The algorithm builds a regressor on the subset of Laplacian embeddings and uses the model to predict the labels for the remainder of the data. An accuracy calculation, similar to the one performed for clustering, is employed to measure the performance of the algorithm.

Algorithm 6 Semi-supervised Regression

Require: The data matrix, $X \in \mathbb{R}^{N\times d}$, Ground-truth labels $Y \in \mathbb{R}^N$, optimal Gaussian width σ^*, the number of features and samples to choose, M features and J samples.
Ensure: Regression accuracy, $accuracy$
1: Get eigen vectors of laplacian embedding matrix $e \leftarrow \text{LaplacianEmbedding}(X, \sigma^*)$
2: Subset the 1st J rows and M features of the eigen vector, $A \leftarrow e[:J, :M]$
3: Subset the 1st J rows of true labels, $b \leftarrow y[:J]$
4: Calculate the solution of the least squares problem, $\hat{\beta} = \text{NormalEquationSolver}(A, b)$
5: Compute the regression value for rest of the data, $z \leftarrow \text{MatrixMultiplication}(e[:, :M], \hat{\beta})$
6: Predict the class by taking the sign of the regression output, $\hat{y} \leftarrow \text{sign}(z)$
7: Calculate misclassified samples, $misclassified = \text{sum}(y \neq \hat{y})$
8: Calculate accuracy $accuracy \leftarrow 1 - (\frac{misclassified}{N})$
9: **return** $accuracy$

4. Computational Results

This section compiles the empirical results of the unsupervised clustering and semi-supervised regression methods applied to the Congressional voting dataset.

4.1. **Spectral Clustering.** The evaluation of the clustering methodology is primarily focused on the sensitivity analysis of Spectral Clustering performance relative to kernel bandwidth.

The robustness of the Spectral Clustering algorithm was evaluated across a parameter sweep of the Gaussian kernel bandwidth, strictly bounded within the domain $\sigma \in [0.1, 4.0]$. As illustrated in Figure 2, the algorithm achieved a peak classification accuracy of $\approx 88\%$ at an optimal bandwidth of $\sigma = 2.1$.

The model exhibits near-random performance when $\sigma < 1.0$. In this lower-bound regime, the exponential decay of the similarity kernel is too aggressive, effectively shattering the graph into disconnected components and preventing the Laplacian from capturing global manifold structure. However, once σ exceeds the critical connectivity threshold, accuracy sharply converges to a stable plateau exceeding 85%. This plateau indicates that for the tested range, the underlying bipartite topological bottleneck is so starkly defined that even under heavy global kernel smoothing, the inter-party edge weights remain sufficiently weaker than intra-party connections, allowing the Laplacian to maintain the correct cut. [1]

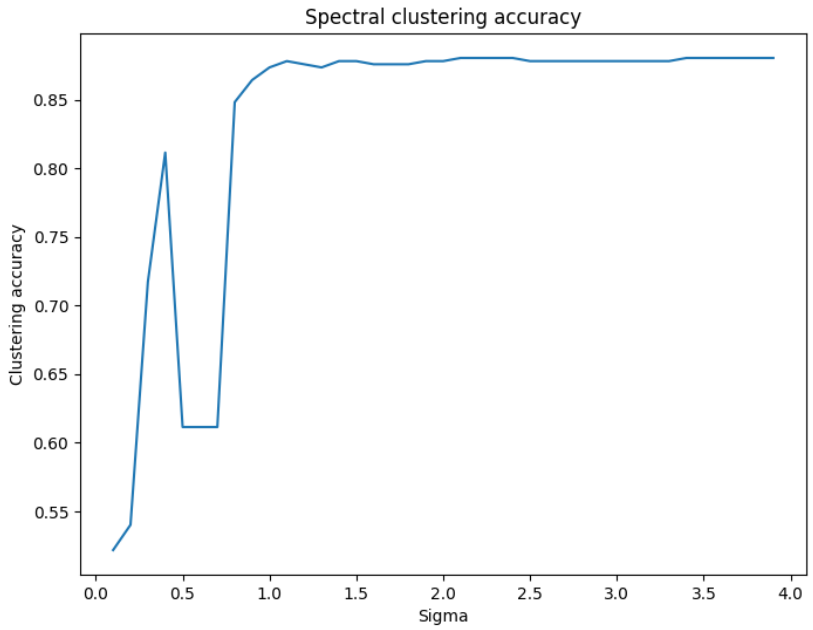


FIGURE 2. Spectral clustering performance by varying hyperparameter σ

4.2. **Semi-supervised Regression results.** The regression algorithm used the optimal Gaussian width ($\sigma = 2.1$) that achieved highest accuracy in Spectral Clustering to build its Laplacian embedding. The embedding built using variable number of feature count and sample size was tested to predict the labels of the entire dataset and its accuracy was measured against the ground truth. Multiple combinations of feature count and sample size achieved high accuracy ($> 87\%$) with the 4 feature to 10 sample combination achieving maximum accuracy of 91.5%. The full list of hyper parameters tested and their accuracy is listed in table 1.

Other interesting observations about the results:

- Fiedler vector still plays an important role.

[1] Empirical testing revealed this plateau persists even at $\sigma = 1e^6$. While theoretical continuous mathematics dictates the graph should eventually fully connect and lose all manifold structure ($w_{ij} \to 1$), the 64-bit floating-point precision of the eigensolver preserves the microscopic relative variations between intra- and inter-cluster distances, maintaining the stability of the Fiedler vector.

Feature count (M)	**Sample size (J)**	**Accuracy**
2	5	87.36%
	10	87.36%
	20	87.82%
	40	87.60%
3	5	89.88%
	10	87.13%
	20	88.05%
	40	87.82%
4	5	39.54%
	10	91.50%
	20	89.89%
	40	89.20%
5	5	41.15%
	10	74.94%
	20	85.06%
	40	86.44%
6	5	64.14%
	10	81.84%
	20	89.42%
	40	86.21%

TABLE 1. Semi-supervised Regression performance per feature count and sample size

  - The accuracy for low feature count (2, 3) are consistently high regardless of sample size. This is because the Fiedler vector already perfectly splits the graph along its weakest fault line and performs most of the heavy lifting for regression model.
- Overfitting when $M \approx J$
  - When the number of features and samples are approximately equal, the accuracy is extremely low. This is because the model tries to fit a 4-D or 5-D plane using only 5 data points. This causes the model to memorize the 5 specific data points and build a model that fails to generalize to rest of the data points. Trying to perform regression on a dataset where $M \approx J$ creates an under-determined system. When degree of freedom is zero, the least square normal equation becomes poorly conditioned causing massive drop in accuracy.
- Higher feature count and sample size does not guarantee higher accuracy
  - The highest feature count and sample size combination (6 features and 40 samples) did not have the highest accuracy, in fact its accuracy was less than all combinations that had 2 or 3 features. This is because the eigenvectors are sorted by their graph connectivity strength. The Fiedler vector has already found the global split and the later vectors only determine noisy local clusters thereby distracting the model from making more accurate predictions.

5. SUMMARY AND CONCLUSIONS

One the common methods in data analysis is to build new bases for existing dataset and project the data onto this bases to extract meaningful information from the data. Graph Laplacians build a map that surfaces the connectivity strength among the data points. Diffusion maps build a new basis using the eigenvectors of the Graph Laplacian. Projecting the dataset onto the new basis naturally manifests the underlying grouping of data.

Spectral clustering is often preferred when the dataset lies a non-linear manifold and is grouped into an unknown number of clusters. Diffusion maps provide insights into the cluster counts and applying standard

k-Means on the Diffusion maps is a standard industry practice to reduce clustering errors that occur due to rigidity of the Euclidean boundaries. In this paper, we explored this approach for a bi partite dataset by basing the decision on the sign [2] of the entries of the Fiedler vector.

Semi-supervised learning leverages the manifold learning mechanism of diffusion maps and uses the top eigenvectors to solve a least squares problem for the subset of the dataset with known labels. This elegant method helps to perform regression or classification on the unlabeled subset of the dataset.

6. Appendix

6.1. **Eigengap Heuristic.** The U.S. Congressional voting dataset has a known number of clusters. The real-world datasets might not exhibit such bi partition behavior and require data driven methods to determine the actual number of meaningful clusters. While the number of clusters (K) for *k-Means* algorithm is often chosen by trial and error methods, Spectral methods provide a deterministic way to determine the cluster count. The eigengap heuristic states that the optimal number of clusters K is found where the gap between λ_K and λ_{K+1} is unusually large, and the eigenvalues after that gap abruptly level off. Figure 3 visually proves that the cluster count $K = 2$ was not due to presence of *two* parties in U.S. congress, but the underlying graph topology proves that exactly *two* massive structures existed in the data.

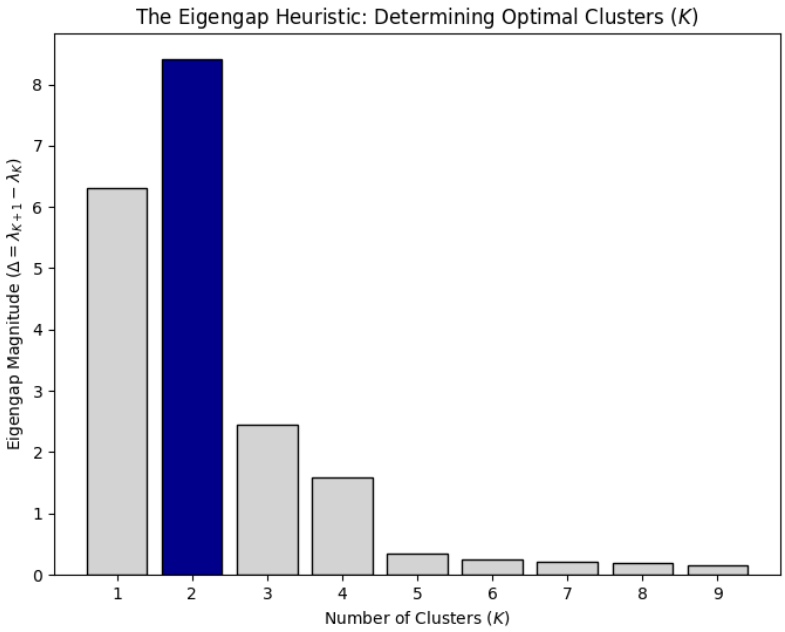


Figure 3. The Eigengap Heuristic $\Delta = \lambda_{K+1} - \lambda_K$. Spectral graph theory dictates that the optimal number of clusters K occurs before the eigenvalue spectrum plateaus. The precipitous drop in eigengap magnitude after $K = 2$ mathematically confirms the bipartite structure of the Congressional voting manifold, independent of the ground-truth party labels.

6.2. **Topological versus Euclidean boundary comparison (*k*-Means vs. Spectral Clustering).** In the theoretical section 2, we claimed the advantage of Spectral Clustering is its ability to utilize the geometry of the data to make an informed decision. To understand the operational differences between the two algorithms, we isolated the specific data points where their predictions diverged (Figure 4). Notably, these disagreements

[2]The sign method is often considered a 1D version of *k-Means* algorithm

did not occur randomly, but localized exclusively along the topological boundary bridging the two primary partisan clusters.

Ground-truth evaluation of this specific 'battleground' subset revealed that Spectral Clustering marginally outperformed *k-Means* (correctly classifying 3 of the 5 contested nodes, compared to 2 by *k-Means*). While this suggests the graph Laplacian's topological approach is slightly more robust at tracing the non-linear allegiances of moderate voters, the tight margin highlights the extreme ambiguity of this boundary space. These specific representatives act as true statistical anomalies whose voting behavior challenges both rigid Euclidean hyperplanes and continuous manifold assumptions.

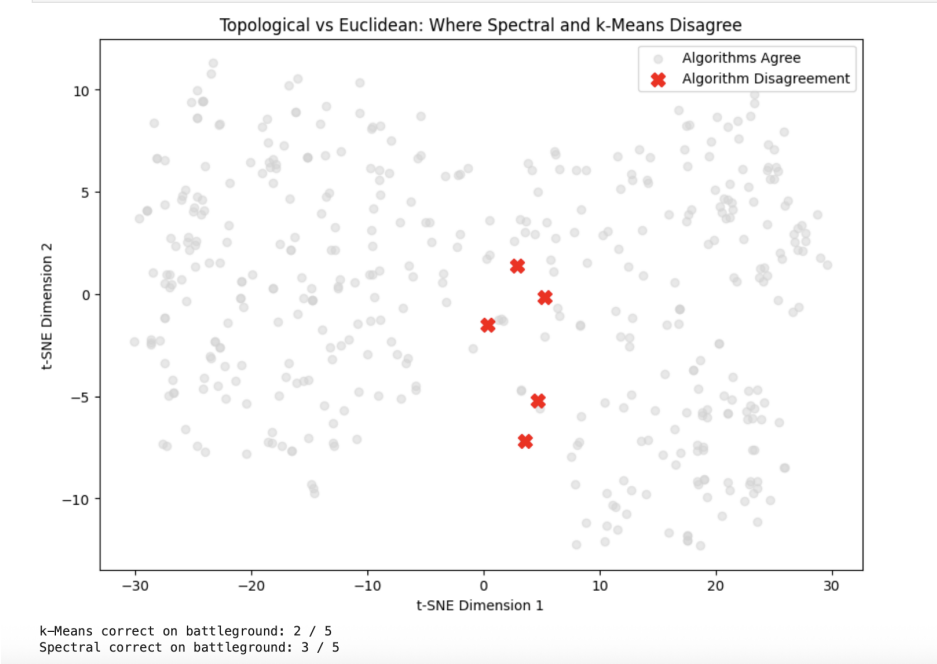


FIGURE 4. Spectral clustering slightly outperforms k-Means along the topological boundary.

7. ACKNOWLEDGMENT

The author expresses deep gratitude to Prof. Bamdad Hosseini for their AMATH-582 lectures covering the core topics on data analysis. The lecture notes were an invaluable resource in writing this paper. The author also would like to extend appreciation to the course TA, Saba Heravi, for their support during office hours and through other platforms.